PROJECT SYNOPSIS

Self-Healing MLOps System with Drift Detection and Auto-Retraining

Using Docker | Jenkins | Kubernetes | Prometheus | Grafana

Project Title	Self-Healing MLOps System with Drift Detection and Auto-Retraining
Branch / Stream	B.Tech – Computer Science / Information Technology
Academic Year	2023–2027
Domain	DevOps / Cloud Computing
Tools & Technologies	GitHub, Docker, Jenkins, Kubernetes, FastAPI, Prometheus, Grafana
Project Type	Individual

1. Introduction

Modern applications powered by machine learning require not only deployment but **continuous monitoring and adaptation**. Traditional systems fail when data changes over time, causing model performance degradation.

This project introduces a **Self-Healing MLOps System** that integrates DevOps practices with machine learning workflows. It automates the entire lifecycle — from model deployment to **drift detection, retraining, and redeployment** — ensuring the system remains accurate and reliable.

2. Problem Statement

Traditional ML deployment faces several challenges:

1. Models degrade due to **data drift**
2. No automatic retraining → outdated predictions
3. Manual deployment → slow and error-prone
4. No monitoring → issues go unnoticed

5. Lack of CI/CD → inconsistent releases

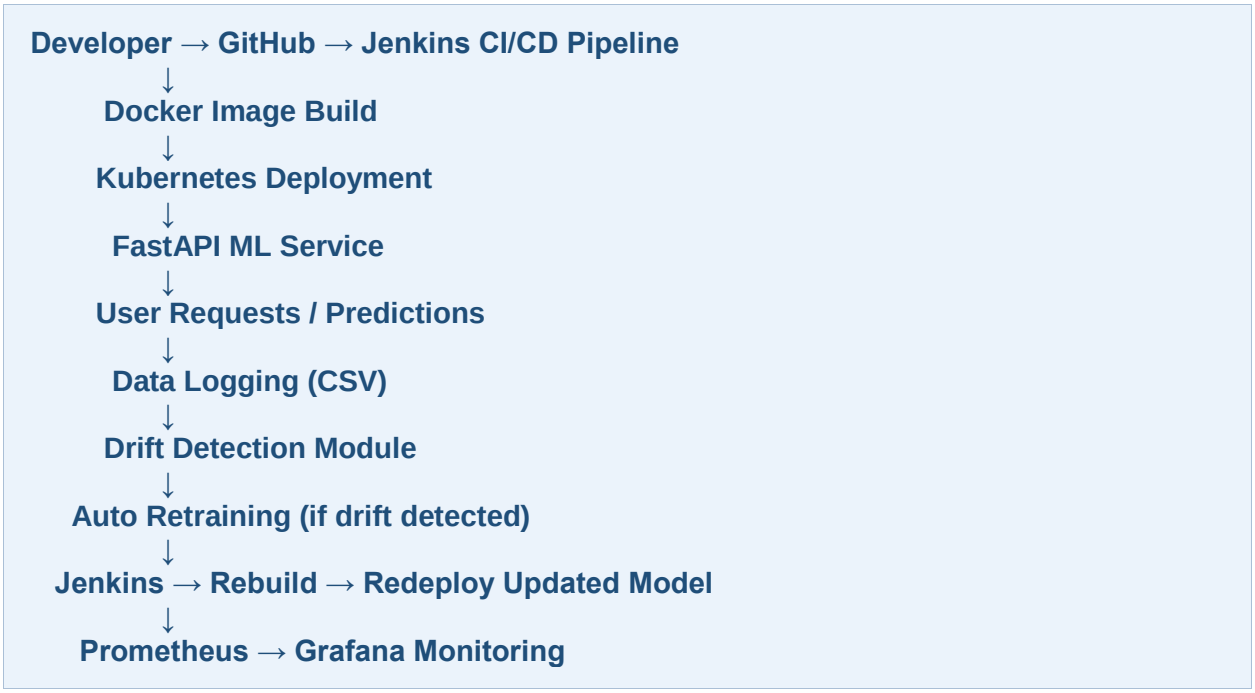
This project solves these using **DevOps + MLOps integration**.

3. Objectives

1. Build and deploy an ML model using FastAPI
2. Containerize the application using Docker
3. Automate build and deployment using Jenkins
4. Deploy application on Kubernetes cluster
5. Implement data logging and drift detection
6. Automatically retrain model when drift is detected
7. Monitor system using Prometheus and Grafana
8. Create a fully automated **self-healing pipeline**

4. System Architecture

The project follows a linear DevOps pipeline:



5. Project Modules

Module 1 – ML Model & API

- Train baseline ML model (churn prediction)
- Build FastAPI service for inference
- Save model as model.pkl

Module 2 – Docker Containerization

- Create Dockerfile
- Package API + model
- Ensure portability across environments

Module 3 – Jenkins CI/CD Pipeline

- Integrate GitHub with Jenkins
- Automate:
  - Code pull
  - Docker build
  - Image tagging
- Extend pipeline for retraining

Module 4 – Kubernetes Deployment

- Deploy application using Deployment + Service
- Manage scaling and availability
- Enable container orchestration

Module 5 – Drift Detection & Retraining

- Log incoming data and predictions
- Compare training vs live data
- Detect drift using statistical methods
- Trigger retraining pipeline automatically

Module 6 – Monitoring & Observability

- Integrate Prometheus for metrics collection
- Use Grafana for dashboards
- Track:
  - Request count
  - Latency

6. Project Plan / Timeline

Phase	Activity	Tools Used	Duration
Phase 1	ML Model Development	Python, Scikit-learn	Week 1
Phase 2	API & Dockerization	FastAPI, Docker	Week 2
Phase 3	CI/CD Setup	Jenkins, GitHub	Week 2
Phase 4	Kubernetes Deployment	Kubernetes, Docker	Week 3
Phase 5	Drift Detection & Retraining	Python	Week 3
Phase 6	Monitoring Setup	Prometheus, Grafana	Week 4
Phase 7	Testing & Documentation	All Tools	Week 4

7. Expected Outcomes

- Fully automated ML deployment pipeline
- Real-time monitoring dashboard
- Automatic detection of data drift
- Self-healing system via auto-retraining
- Reduced manual intervention
- Production-ready DevOps workflow

8. Real-World Applications

- E-commerce recommendation systems
- Fraud detection systems in banking
- Customer churn prediction models
- AI-based SaaS platforms
- Any ML system requiring continuous learning.

9. Conclusion

This project demonstrates how **DevOps and MLOps together** create a powerful system that not only deploys machine learning models but also maintains and improves them over time.

By integrating tools like **Docker, Jenkins, Kubernetes, Prometheus, and Grafana**, the system becomes scalable, automated, and production-ready.

It reflects real-world industry practices used by companies for **continuous delivery and intelligent automation**, making it highly relevant for careers in DevOps, MLOps, and Cloud Engineering.

References

- Docker Documentation – https://docs.docker.com
- Jenkins Documentation – https://www.jenkins.io/doc
- Kubernetes Docs – https://kubernetes.io/docs
- Prometheus Docs – https://prometheus.io/docs
- Grafana Docs – https://grafana.com/docs